CS5013: Programming with AI — Principles & Practices Document

GUI Framework for Multi-Tool Panoramic Image Segmentation

Student: Rathan Balaji P R | **Roll No:** CS26E009 | **Stakeholder / Faculty:** Dr. Sukhendu Das, CSE, IIT Madras

Synthesis: Hands-on Labs 1–3, Exam Master Guides & Project Lifecycle Implementation

1. System Overview & Core Engineering Challenges

Panoramic semantic segmentation assigns pixel-level categories to stitched 360° equirectangular panoramas. The domain introduces severe geometric distortions (360° pole stretching, aspect ratios up to 12:1) and runtime fragmentation (PyTorch/CUDA, C++, REST APIs). To unify heterogeneous tools within a single desktop application, we engineered a modular **Java 21 + JavaFX** application adhering to four core architectural pillars: (i) **Adapter Pattern & Isolation Boundary:** external deep learning models execute in isolated subprocesses via Java ProcessBuilder, shielding the JVM from native crashes; (ii) **Canonical Schema Normalization:** heterogeneous mask formats map into an immutable SegmentationResult; (iii) **Panorama-Aware View:** interactive 360° cyclic horizontal modulo yaw rotation eliminating 0°/360° seam artifacts; and (iv) **Metrics Engine:** evaluating Pixel Accuracy, mIoU, and Dice against Ind-PanSeg ground truth.

2. Prompt Engineering: Principles, Techniques, & Course Mappings

Across the project lifecycle, prompt engineering was treated as a rigorous software specification discipline rather than ad-hoc chat, applying the core principles mastered in Hands-on Sessions 1.1–2.4:

- **A. Role-Prompting & Format Constraints (Hands-on 2.2 IntentParser):** Hands-on 2.2 proved that Zero-Shot prompts produce conversational preamble noise (*'Sure, here is the JSON...'*) on 50% of runs, breaking automated parsers. By enforcing Role + Explicit Output Constraints (*'You are a strict parsing service. Output ONLY JSON'*), preamble noise dropped to 0%. In our project, all tool interchange interfaces (e.g. tools.json, model runners) strictly enforce role-framing and zero-preamble JSON output schemas, guaranteeing 100% parseable inter-process communication.
- **B. Context Curation & Open Tab Heuristic (Hands-on 3.1 UserDTO):** Hands-on 3.1 demonstrated that Fill-In-The-Middle (FIM) context fails when relevant files are closed, causing the LLM to hallucinate fields. Opening adjacent tabs cured the hallucination. In our workflow, we systematically curated prompt context by grounding the model in the authoritative CS5013 Proposal, Design Document, and ICVGIP 2026 *PanoFusion* paper before code generation, eliminating contract drift.
- **C. Specificity, Invariants, & Memory Optimization (Hands-on 2.1 Dedup):** Hands-on 2.1 showed that vague prompts (*'Make it faster'*) violate invariants (HashSet lost insertion order), while specific prompts enforcing O(n) order preservation yielded LinkedHashSet (6 ms), and iterative follow-ups yielded BitSet for near-zero memory. We applied this by explicitly specifying mathematical invariants: e.g. equirectangular modulo arithmetic (x + offset) % width and strictly immutable Java Records/Builders, preventing unsafe optimizations.
- **D. Prompt Chaining vs One-Shot Refactoring (Hands-on 2.3 InvoiceReport):** Hands-on 2.3 proved that a 4-step chain (**Plan** → **Generate** → **Critique** → **Patch**) outperforms one-shot refactoring by catching unnecessary indirection and missing validations. We used prompt chaining to develop our PanoramaImageRenderer and MetricsCalculator, allowing the AI to critique its initial edge-case handling before generating final production code.
- **E. Explicit Capitalized Negative Constraints (Hands-on 2.3 Negative Prompting):** Course findings showed that negative prompts must be capitalized and explicit. We enforced critical guardrails: *'Do NOT embed Python/CUDA inside JVM'*, *'Do NOT drop unmapped tool metadata; store in unsupportedAttributes Map'*, and *'Do NOT invent arbitrary classes; use exact 21 Ind-PanSeg classes'*.

3. Debugging: Structured Approach & Issue Resolution

Debugging across a hybrid JavaFX-Python-CUDA environment required a disciplined four-step diagnostic methodology:

• **1. Boundary Isolation & Subprocess Decoupling:** When external segmentation failed, we decoupled the Java layer from Python. We verified scripts standalone via terminal CLI on test panoramas before connecting ProcessModelAdapter, preventing cross-boundary diagnostic confusion.

• **2. Deadlock-Free Stream Handling:** A classic defect in external process management is OS pipe buffer deadlock when stdout/stderr streams fill. We implemented dedicated background worker threads reading stdout and stderr concurrently via BufferedReader, preventing Java execution freezes.

• **3. Deterministic Mock Baseline as Oracle:** To debug rendering and metric calculations without GPU or network latency, we authored DeterministicTestAdapter. It synthesizes mathematically predictable horizontal wrap-around regions, proving the metric formulas and modulo yaw rotation independently.

• **4. Test-Driven Reproducers & Characterization Tests:** Following course teachings (Michael Feathers), before refactoring the toy 6-class schema to the 21-class Ind-PanSeg benchmark, characterization tests froze existing behavior. When schema mismatches occurred, focused reproducer tests in AdapterTest.java identified missing keys immediately.

4. Version Control (Git): Code & Prompt Lifecycle Tracking

Version control was utilized as a rigorous audit trail linking code commits, architectural decisions, and AI prompts according to course standards:

• **A. 'The WHY is Yours to Write' (Hands-on 3.1 Part D Core Rule):** As established in Hands-on 3.1, AI tools can generate commit subjects by inspecting git diff --staged (WHAT changed), but the developer must author the commit body to record WHY the change was made (e.g., defensive architectural boundary, regression prevention, or benchmark compliance). All 16+ commits in our repository include detailed rationale bodies explaining engineering decisions.

• **B. Conventional Commits & Atomic Progression:** We adhered strictly to Conventional Commits: feat: (features), test: (test suites), docs: (specifications), and refactor: (structural improvements). Every commit represents an atomic, compiling, and fully passing state (e.g. feat: integrate PanoFusion and Ind-PanSeg 21-class models from ICVGIP 2026 paper and test: add 35 comprehensive JUnit 5 tests).

• **C. Bidirectional Prompt-to-Commit Traceability:** Functional changes in Git are linked directly to prompts stored in version-controlled markdown logs (PROMPTS.md and transcripts). This establishes complete pedagogical reproducibility as mandated by the CS5013 evaluation rubrics.

• **D. Repository Hygiene & Secret Prevention (Hands-on 1.3 SecretSweep):** Hands-on 1.3 highlighted that hardcoding credentials or system paths leaks secrets to version control and training logs. In our project, all commands, paths, and timeouts are dynamically parameterized in config/tools.json or loaded via environment variables, with OS files and bytecode strictly ignored via .gitignore.

5. Testing, Refactoring, & Documentation Standards

Software quality and maintainability were enforced through multi-tiered testing, GoF design patterns, and defensive Java patterns:

Test Suite Tier	Test Classes & Coverage Target	Course Verification Principle
Unit Tests	MetricsCalculatorTest, ResultNormalizerTest, SegmentationModelTest	AAA Pattern (Arrange-Act-Assert); verified mIoU, Dice, Pixel Accuracy against hand-calculated oracle values.
Boundary & Defensive Tests	ExecutionBoundaryTest, ValidationServiceTest	Tested timeout enforcement, invalid 15:1 aspect ratios, corrupted headers, and stream buffer isolation.
Integration & E2E Tests	AdapterTest, AsyncCoordinatorTest, GuiWorkflowTest, EndToEndSystemTest	Complete async multi-tool pipeline: validation → Python execution → normalization → wrap-around view → metrics.

• **Refactoring to GoF Design Patterns:** 1. **Adapter Pattern:** ProcessModelAdapter adapts heterogeneous runtimes into the common SegmentationTool contract.
2. **Strategy Pattern:** Metrics calculation and image rendering algorithms are decoupled from model execution.
3. **Observer/Listener Pattern:** AsyncSegmentationCoordinator updates JavaFX views asynchronously without blocking UI.
4. **Builder Pattern:** SegmentationResult.Builder guarantees immutability and thread safety.

• **Modern Java 21 Idioms:** Used Java Records (public record MaskData(...)) for boilerplate-free immutable data carriers, preventing AI equals/hashCode errors; leveraged Java Streams API (.filter(), .map()) for declarative class filtering; and applied defensive exception handling without swallowing.

6. Appendix: Concrete Sample Prompts & Engineering Analysis

Below are three representative prompts used during the project lifecycle, demonstrating the principles taught in CS5013:

Sample Prompt 1: Initial Architecture Scaffolding & Negative Constraints

```
Role & Context: You are a Principal Java Architect implementing the CS5013 Design Document for IIT Madras.
Task: Implement the complete domain model and adapter contracts in Java 21.
Constraints & Invariants:
1. Domain entities (SegmentationResult, MaskData, SemanticClass) must be strictly immutable via Builder pattern.
2. External models must NEVER run in the JVM; design ProcessModelAdapter using ProcessBuilder with background stdout/stderr threads.
3. If external models return unrecognized metadata attributes, do NOT drop them. Store them in an unsupportedAttributes Map.
4. Provide JUnit 5 unit tests verifying all model constraints.
Principles Applied: Role-Framing (Hands-on 2.2) + Explicit Capitalized Negative Constraints (Hands-on 2.3) + Builder Immortality.
```

Sample Prompt 2: Cylindrical Modulo Wrap-Around Rendering (Algorithmic Invariant)

```
Context: The input is an equirectangular 360° panorama. Standard 2D viewers fail because objects split across 0° and 360° appear severed.
Task: Create PanoramaImageRenderer in JavaFX to support interactive horizontal yaw rotation.
Algorithmic Invariants & Verification:
Implement cyclic horizontal modulo offset: sourceX = (x + offsetPixels) % width.
Ensure seamless interpolation across the seam. Write a unit test in PanoramaVisualizationTest verifying that pixel labels at x=0 and x=width-1 maintain topological continuity under a 180° shift.
Principles Applied: Mathematical Invariant Specification (Hands-on 2.1) + Domain Geometry Grounding + Test-Driven Characterization.
```

Sample Prompt 3: Integrating ICVGIP 2026 Paper Benchmark Models (Context Curation)

```
Context: Grounded in research paper: 'PanoFusion: Boundary-Aware Vision-Language Segmentation for Stitched Indoor Panoramas' (ICVGIP 2026).
Task: Integrate the exact models from Table 3 into config/tools.json and external runners:
1. panofusion: CLIP + DAN + SBR-Net MobileSAM boundary refinement (54.94% mIoU).
2. ops-baseline: Coarse vision-language backbone without SBR-Net (44.12% mIoU).
3. segformer-baseline: Perspective transformer fine-tuned with focal loss (31.35% mIoU).
4. Adopt the authoritative 21 classes of Ind-PanSeg (Table 2: Roof, Bed, Wall, Floor, Monitor, Chair, Table, etc.).
5. Update test assertions and verify all 35 tests pass with zero regressions.
Principles Applied: Context Curation / Open Tab Heuristic (Hands-on 3.1) + Zero Architecture Modification (Pure GoF Adapter Extension).
```

7. Comprehensive Mapping to Course Exam Concepts

Course Module / Topic	Hands-on Exercise	How Applied in this Project Lifecycle
M1: Secret Hygiene	1.3 SecretSweep (System.getenv)	No credentials or hardcoded OS paths; dynamic JSON and env parameterization.
M2: Format Constraints	2.2 IntentParser (Role + JSON)	Zero conversational preamble; strict JSON interchange schemas across tools.
M2: Negative Constraints	2.3 InvoiceReport (Do NOT...)	Enforced JVM isolation boundary; prohibited loss of unmapped metadata.
M3: Context Curation	3.1 UserDTO (Open Tab Heuristic)	Grounded AI in Proposal, Design Doc, and ICVGIP 2026 paper, eliminating hallucinations.
M3: AI-Assisted Git	3.1 Part D ('The WHY is Yours')	16+ atomic commits where commit bodies record the engineering WHY behind each change.
Testing: AAA Pattern	JUnit 5 Guide (Michael Feathers)	35 automated tests freezing characterization behavior before refactoring 21 classes.

Student Declaration: I confirm that all code, tests, and documentation reflect my own architecture and understanding, guided by software engineering principles taught in CS5013. AI tools were utilized strictly in compliance with course policy.